University of Applied Sciences and Arts Northwestern Switzerland
School of Business

member of swissuniversities
AACSB ACCREDITED

Advanced Programming

Object Oriented Programming (OOP)

Today's Topics

Programming Paradigms

- From Procedural Programming
- To Object-Oriented Programming (OOP)

Core Concepts

- Objects and Classes
- Attributes: Object State (data)
- Methods: Object Behavior (functionality)

Object Creation

- Construction (`__new__`)
- Initialization (`__init__`)

Encapsulation

- Encapsulation in Python
- Getters and Setters

Object Interaction

- References: How objects interact

Python Object Model

- Magic Methods

Design Principles

- Single Responsibility Principle (SRP)

Procedural Programming

```python
def create_student(student_id: str, fname: str, lname: str):
    return {
            "student_id": student_id,
            "firstname": fname,
            "lastname": lname,
            "modules": []
    }

if __name__ == '__main__':
    stephan = create_student("25875567", "Stephan", "Müller")
    print(stephan["student_id"])
    print(stephan["firstname"])
    print(stephan["lastname"])
```

What might be the problem?

- No structural guarantees
- Dictionaries are error-prone (e.g., typos, missing keys)

What happens if a required key is missing?

Procedural Programming

```python
def enroll(student, module):
    if module not in student["modules"]:
        student["modules"].append(module)

if __name__ == '__main__':
    programming = create_module("Programming")
    stephan = create_student("25875567", "Stephan", "Müller")

    enroll(stephan, programming)
```

What might be the problem?

- Data and behavior are separated
- No clear abstraction boundary

This leads to

- Loosely coupled functions
- Data passed throughout the system
- Reduced maintainability

Functions depend on the internal structure of the data without formally enforcing it.

For example, the key `"modules"` must be initialized explicitly when creating a student dictionary.

Procedural Programming

The philosophy of procedural programming was:

- Structure the program as sequence of instructions packed in functions
- The focus is on functions that operate on data
- Data and functions are typically separated
- Problems are solved step by step through function calls

Object-Oriented Programming

Philosophy of Object-Oriented Programming (OOP):

- Model software after real-world objects (Such as Student, Professor, Module, Book, etc.)
- Combine data and behavior into meaningful objects
- Mirror how humans naturally understand the world
- Improve readability, structure, and maintainability

This leads to:

- Better understanding
- Easier maintenance
- Improved overview of complex systems

Procedural vs. Object-Oriented Programming

Procedural	OOP
Data often in basic data structures	Structured objects defined by a class
Functions are separated from data	Data and functionality are close together
No structural guarantees	Constructor initializes structure
Easy to misuse	Clear interface
Harder to scale	Scales naturally

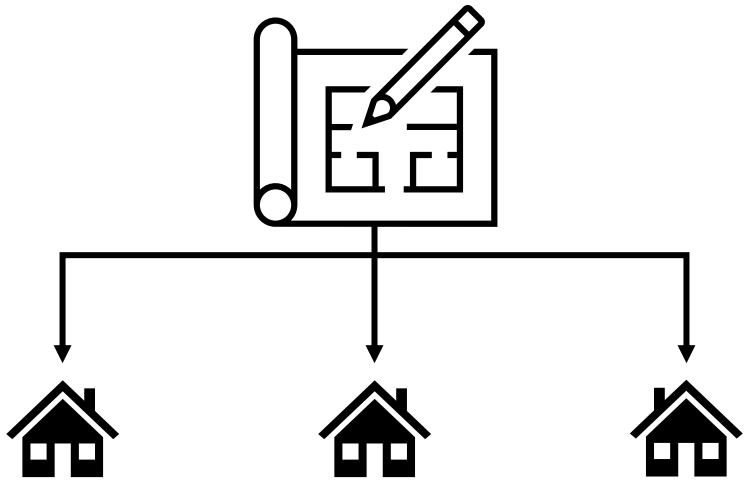
Objects and Classes

Similar to a building blueprint, one blueprint can be used to build multiple houses. All these houses share the same structure, but they may differ in wall colors, kitchen appliances, or furniture.

Each individual house represents an instance (object) created from the same blueprint (class).

Class: The blueprint of a house

Object: An instance of a house built according to the blueprint



Classes

A class defines:

- **Attributes:** define the object's structure or state (data)
- **Methods:** define the object's behavior (functions)

Objects are created from this class to represent individual instances of the same type.

Similar to a blueprint of a house, the engineer must decide how detailed the plan has to be to achieve the goal. You don't need to model each individual brick to build a house but if you need special features in a building, you better model the details good enough that the constructors on site can build that feature.

Class Example: Student

What are possible attributes (data) representing a student?

- first name, last name
- address
- birthday
- matriculation number
- modules
- …

What are possible methods (behavior) a student might have?

- enroll / deregister
- take exam
- pay tuition
- generate transcript of records
- …

Example Class

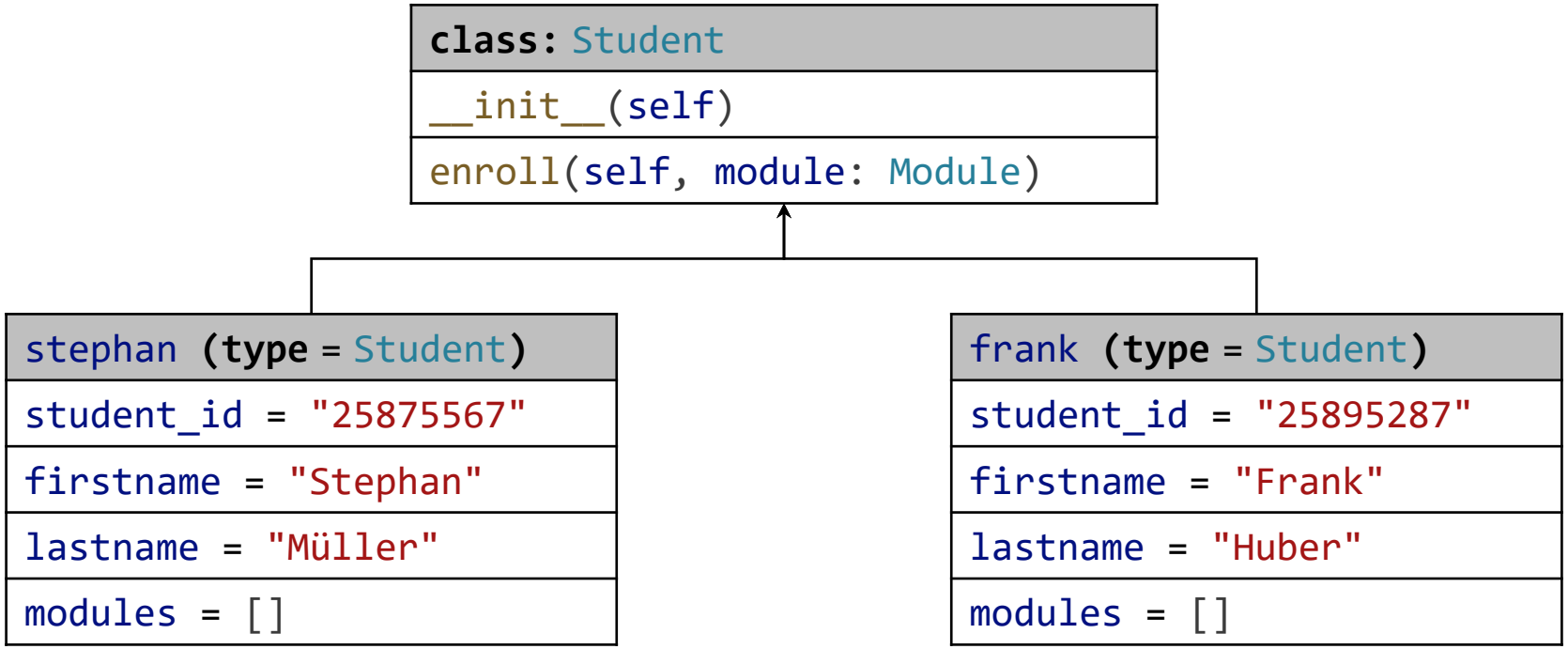
```python
class Student:
    def __init__(self, student_id: str, firstname: str, lastname: str) -> None:
        self.student_id: str = student_id
        self.firstname: str = firstname
        self.lastname: str = lastname
        self._modules: list[Module] = []

    def enroll(self, module: Module) -> None:
        if module not in self._modules:
            self._modules.append(module)


if __name__ == '__main__':
    module = Module("Programming")
    stephan = Student("25875567", "Stephan", "Müller")
    stephan.enroll(module)
```

Objects

```
stephan = Student("25875567", "Stephan", "Müller")
frank = Student("25895287", "Frank", "Huber")
```



Instance Attributes: The states (object data)

- Instance attributes are defined at **object level**
  - store an object's **state**
  - object can have its own values

- Instance attributes are usually assigned in the initialization of an object (`__init__`)

```
class Student:
    def __init__(self, student_id: str, firstname: str, lastname: str) -> None:
        self.student_id: str = student_id
        self.firstname: str = firstname
        self.lastname: str = lastname
        self._modules: list[Module] = []
```

Class Attributes: The shared state (class data)

- Class attributes are defined at **class level** and shared by **all instances**
  - store **class-level/shared state**
  - belong to the **class itself**, not to individual objects

- E.g. count how many student objects are created to know how many students there are

class level: not in `__init__`, but within the class

```
class Student:
    _counter = 0

    def __init__(self, student_id: str, firstname: str, lastname: str) -> None:
        self.student_id: str = student_id
        self.firstname: str = firstname
        self.lastname: str = lastname
        self._modules: list[Module] = []
        Student._counter += 1
```

class level: not `self`, but `Student`

Construction: __new__

The constructor (__new__) is responsible for creating a new instance (object) of a class.

Pythons internal object creation process:

```
stephan = Student.__new__(Student, "25875567", "Stephan", "Müller") #creates the new object
Student.__init__(stephan, "25875567", "Stephan", "Müller") #initializes the new object
```

Use custom __new__ only for special cases:

- immutable types
- singleton/factory patterns
- instrumentation

Initialization: `__init__`

The initializer (`__init__`) is responsible for setting up the state of a newly created object.

```
stephan = Student("25875567", "Stephan", "Müller")
```

```
class Student:
    def __init__(self, student_id: str, firstname: str, lastname: str) -> None:
        self.student_id: str = student_id
        self.firstname: str = firstname
        self.lastname: str = lastname
        self._modules: list[Module] = []
```

It does **not create** the object but receives the instance created by `__new__` and assigns attributes.

`__new__` vs. `__init__`

`__new__` constructs a new object

`__init__` initializes the state of the object

Execution order: `__new__` → `__init__`

Method	Responsibility	Returns	Level	First Param
`__new__`	Creating object	An instance of the class (object)	Class	`cls`
`__init__`	Initializing object	None	Instance	`self`

⚠ Most classes do not override `__new__`,
but almost always `__init__`.

Parameter: `self`

What is self?

- `self` is a reference to the instance.
- It allows access to the object's attributes and methods.
- It is passed automatically by Python.

`self` is not a keyword!
It is just a naming convention for the first parameter of instance methods.

```
stephan = Student("25875567", "Stephan", "Müller")
```

```
class Student:
    def __init__(self, student_id: str, firstname: str, lastname: str) -> None:
        self.student_id: str = student_id
        self.firstname: str = firstname
        self.lastname: str = lastname
        self._modules: list[Module] = []
```

Instance Methods: The behavior (functionality)

- Instance methods define the **behavior** of an object.
- They operate on the object's **state**.
- Instance methods always receive the instance as the first parameter (`self`).

```
stephan.enroll(module)
```

```
class Student:
    …
    def enroll(self, module: Module) -> None:
        if module not in self._modules:
            self._modules.append(module)
```

Methods are like **functions** that belong to a **class** and **operate** on **instances**.

Class Methods: Behavior related to the class (functionality)

- Class methods operate on the **class itself**, not on a specific object.
- They operate on the class's **state**.
- Class methods always receive the class as the first parameter (`cls`).

```
Student.student_count()
```

```
class Student:
    _counter = 0
    …
    @classmethod
    def student_count(cls) -> int:
        return cls._counter
```

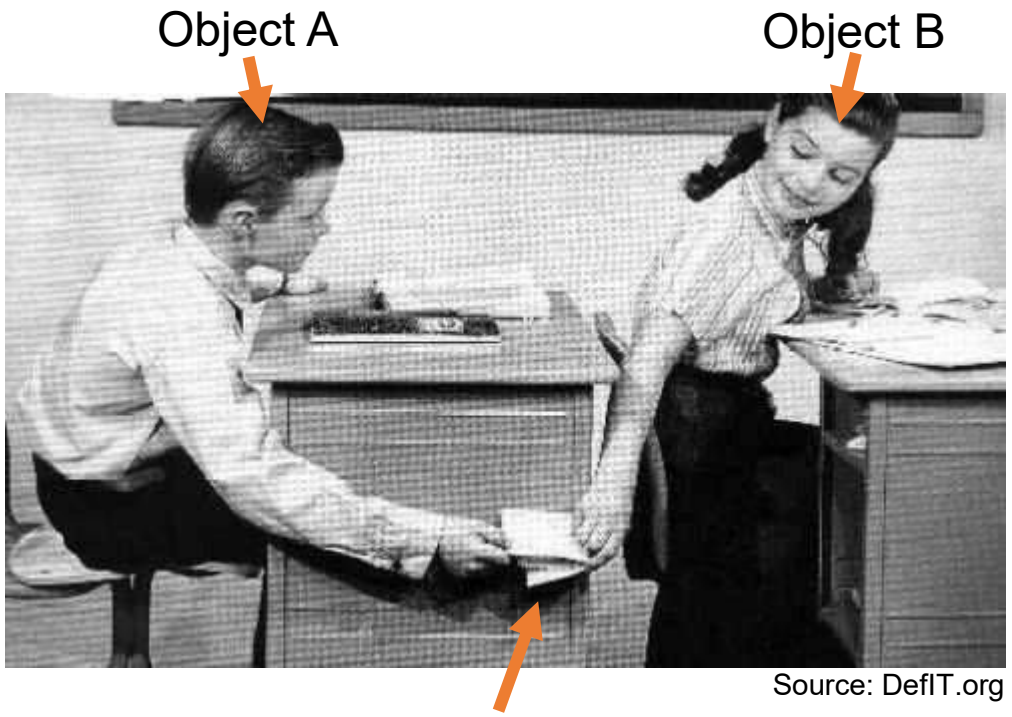
Class methods are like **functions** that belong to a **class** and **operate** on the **class-level**.

How objects interact

A method call is a message sent from one object to another.

Object A → `stephan.enroll(module)` → Object B

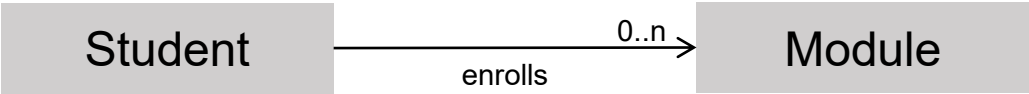
```
class Student:
    …
    def enroll(self, module: Module) -> None:
        if module not in self._modules:
            self._modules.append(module)
```



Source: DefIT.org

Relations

Relationships define how objects reference each other.



- Does Module also know its students?
- Is the relation bidirectional?
- Or is the relation unidirectional?

```
class Student:
    …
    def enroll(self, module: Module) -> None:
        if module not in self._modules:
            self._modules.append(module)
```

Getters and Setters

`@property`

- Cheap (no computation, no blocking)
- Predictable (same result for same state)
- Side-effect free (no state change)
- Attribute-like (just return a state)

```
class Student:
    …
    @property
    def student_id(self) -> str:
        return self._student_id
```

Private

`@property_name.setter`

- Cheap (no computation, no blocking)
- Deterministic (validation only)
- Local state change only

⚠ A `setter` cannot exist without a `property`.

```
class Student:
    …
    @student_id.setter
    def student_id(self, value: str) -> None:
        if not value.isdigit():
            raise ValueError("...")
        self._student_id = value
```

Private

Encapsulation: Example

```
class Student:
    def __init__(self, student_id: str, firstname: str, lastname: str) -> None:
        self.student_id: str = student_id
        self.firstname: str = firstname
        self.lastname: str = lastname
        self._modules: list[Module] = []

    @property
    def modules(self) -> tuple[Module, ...]:
        # return a read only copy as tuple, to ensure enrollment via enroll method
        return tuple(self._modules)

    def enroll(self, module: Module) -> None:
        if module not in self._modules:
            self._modules.append(module)
```

Private attribute

Public Interface immutable

Public Method to change state

Encapsulation

What is Encapsulation?

- Bundling data and behavior together and defining a clear public interface.
- Encapsulation is about **controlling how objects are used**, not about hiding everything.
- Expose behavior (methods). Protect structure (data).

Public

- What other objects are allowed to use must be stable and documented

Private

- Marked with a leading underscore: `_attribute`

Privacy is not enforced in Python!
"We are all consenting adults!"

Name mangling

- Two leading underscores leads to name mangling.
- `__attribute` → `_Class__attribute`

Rarely needed: prevents subclass collisions, not access (no privacy).

Magic Methods

– `__str__`: Defines the human-readable string representation of an object.

```
def __str__(self) -> str:
    return f"{self.firstname} {self.lastname}"
```

– `__repr__`: Defines the unambiguous string representation of an object used for debugging.

```
def __repr__(self) -> str:
    return (
        f"Student(student_id={self.student_id!r}, "
        f"firstname={self.firstname!r}, "
        f"lastname={self.lastname!r})"
    )
```

Magic Methods

– `__eq__`: Defines when two objects are considered equal.

```
def __eq__(self, other: object) -> bool:
    if not isinstance(other, Student):
        return NotImplemented
    return self.student_id == other.student_id
```

– `__lt__`: Defines the “less than” (<) relationship between objects.

```
def __lt__(self, other: object) -> bool:
    if not isinstance(other, Student):
        return NotImplemented
    return (self.lastname, self.firstname) < (other.lastname, other.firstname)
```

– Further Magic Methods: `__le__` (<=), `__gt__` (>), `__ge__` (>=), `__len__` (length)
Autogenerated by implementing `__lt__` and annotating the class with `@total_ordering`

Object Identity vs Equality

– Two objects can be equal (`==`) without being the same object (`is`)

```
def __eq__(self, other: object) -> bool:
    if not isinstance(other, Student):
        return NotImplemented
    return self.student_id == other.student_id
```

```
stephan = Student("25875567", "Stephan", "Müller")
berta = Student("25875567", "Berta", "Schmidt")
print(berta == stephan)  # True because the student ID is the same
print(berta is stephan)  # False because they are different objects in memory

print(id(stephan)) # Prints the memory address of the stephan object
print(id(berta))   # Prints a different memory address of the berta object
```

Responsibilities

One Class – One Clear Responsibility

Single Responsibility Principle (SRP)

– A class should have one reason to change.

```
class Student:
    def enroll(self, module: Module) -> None:
    def deregister(self, module: Module) -> None:
```

– Responsibility:

→ Manage student-related state and behavior.

Bad Design (Too Many Responsibilities)

```
class Student:
    def save(…) -> None:
    def load(…) -> None:
    def send_mail(…) -> None:
    def generate_pdf(…) -> None:
    …
```

Key Takeaways

Classes and Objects

Classes define structure and behavior while objects are instances of these structures and behaviors.

Attributes and State

Objects maintain their own state through instance attributes.

Methods

Instance methods operate on object state (`self`).
Class methods operate on class-level state (`cls`).

Encapsulation

Expose a stable public interface while protecting internal structure.

Magic Methods

Special methods (`__str__`, `__repr__`, …) integrate objects into Python's language behavior.

Object Identity vs Equality

`==` compares logical equality

`is` compares object identity.

Single Responsibility Principle (SRP)

A class should have one clear responsibility.